Gradient Boosting

The GBM Framework · XGBoost · LightGBM

A Study Handbook for Python for Data Science

Builds on: Decision Trees & Random Forests

What you will learn

- How boosting differs from bagging (Random Forest) at a conceptual level
- The gradient boosting framework: fitting trees to residuals/gradients, stage by stage
- What XGBoost adds on top of plain GBM: regularization, second-order gradients, smart missing-value handling
- What LightGBM adds for speed and scale: leaf-wise growth, histogram binning, GOSS, EFB
- How to tune the key hyperparameters and use early stopping to avoid overfitting

Example dataset: `titanic.csv` (891 passengers, target: `Survived`)

Contents

1 From Bagging to Boosting **2**

2 The Gradient Boosting Framework **3**
2.1 Additive Modelling . . . 3
2.2 Fitting to the Gradient of the Loss . . . 3
2.3 The Learning Rate / Shrinkage Trade-off . . . 3

3 XGBoost: Extreme Gradient Boosting **4**
3.1 1. Second-Order Gradients . . . 4
3.2 2. Explicit Regularization . . . 4
3.3 3. Smart Handling of Missing Values . . . 4
3.4 Code . . . 4
3.5 Key Hyperparameters . . . 5

4 LightGBM: Light Gradient Boosting Machine **6**
4.1 1. Leaf-wise (Best-first) Tree Growth . . . 6
4.2 2. Histogram-Based Splitting . . . 6
4.3 3. GOSS and EFB . . . 6
4.4 Code . . . 6
4.5 Key Hyperparameters . . . 7

5 XGBoost vs. LightGBM vs. Random Forest **8**

6 Early Stopping and Evaluation **9**
6.1 Why Early Stopping Matters . . . 9
6.2 Evaluating the Final Model . . . 9

7 Practice Questions **10**

1 From Bagging to Boosting

In the previous handbook, Random Forest combined many **independent** trees trained in parallel on random subsets of data and features, then averaged their votes (**bagging**). **Boosting** takes a very different approach: it builds trees **sequentially**, where each new tree is trained specifically to fix the mistakes of the ensemble built so far.

	Bagging (Random Forest)	**Boosting (GBM family)**
Trees trained	In parallel, independently	Sequentially, each depending on the last
Each tree sees	A bootstrap sample of the full data	The *errors* (residuals/gradients) left by prior trees
Combines by	Majority vote / averaging	Weighted sum of all trees' outputs
Main goal	Reduce **variance**	Reduce **bias**, then control variance via regularization

Key Idea

Boosting turns many **weak learners** (shallow trees, often just a few levels deep) into one **strong learner** by having each new tree focus on what the ensemble has gotten wrong so far.

Intuition

Think of a team of tutors correcting a student's exam, one after another. The first tutor grades the exam and finds the main mistakes. The second tutor doesn't re-grade from scratch — they only focus on fixing what the first tutor missed. The third focuses on what's *still* wrong after tutors one and two. After enough rounds, the combined correction is very accurate, even though each individual tutor only fixed a small piece.

2 The Gradient Boosting Framework

2.1 Additive Modelling

Gradient boosting builds the final prediction as a sum of many small trees:

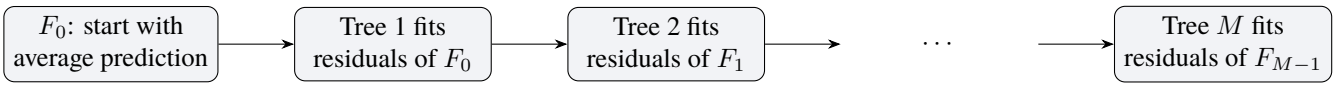
$$F_M(x) = F_0(x) + \sum_{m=1}^{M} \eta \cdot h_m(x)$$

where F_0 is a simple starting prediction (e.g. the average survival rate), each $h_m(x)$ is a small **regression tree** fit at stage m, and η (the **learning rate**) shrinks each tree's contribution so no single tree dominates.

2.2 Fitting to the Gradient of the Loss

At each stage, instead of re-fitting the labels directly, GBM fits a new tree to the **negative gradient** of the loss function with respect to the current predictions — for squared error this residual is simply $y_i - F_{m-1}(x_i)$; for the log loss used in classification it is the gap between the actual label and the predicted probability.

$$r_i^{(m)} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$$



Note

This is why it is called **gradient** boosting: each tree is trained to point in the direction that most reduces the overall loss, the same way gradient descent takes a step in the direction of steepest loss reduction — except here the "step" is an entire decision tree, not a simple parameter update.

2.3 The Learning Rate / Shrinkage Trade-off

- Small η (e.g. 0.01–0.05): each tree contributes only a little, so more trees (`n_estimators`) are needed, but the model generalizes better
- Large η (e.g. 0.3): fewer trees needed, but the model can overshoot and overfit faster
- In practice: pick a small learning rate and let **early stopping** decide how many trees are actually needed (Section 5)

Watch Out

`n_estimators` and `learning_rate` are joined at the hip. Lowering the learning rate without increasing the number of trees just produces an *underfit* model that never gets to express its full capacity.

```python
from sklearn.ensemble import GradientBoostingClassifier

gbm = GradientBoostingClassifier(
    n_estimators=300, learning_rate=0.05, max_depth=3, random_state=42
)
gbm.fit(X_train, y_train)
y_pred = gbm.predict(X_test)
```

3 XGBoost: Extreme Gradient Boosting

XGBoost ("Extreme Gradient Boosting") is an engineered, regularized implementation of the GBM idea that became the dominant algorithm in tabular ML competitions. It keeps the same additive-tree, gradient-fitting framework, but adds three big ideas.

3.1 1. Second-Order Gradients

Plain GBM only uses the first derivative (gradient) of the loss to decide how to fit each tree. XGBoost also uses the **second derivative** (Hessian), giving each tree a more precise target — similar to how Newton's method converges faster than plain gradient descent by also using curvature information.

$$\text{Obj}^{(m)} \approx \sum_{i=1}^{n} \left[g_i h_m(x_i) + \tfrac{1}{2}\eta_i\, h_m(x_i)^2 \right] + \Omega(h_m)$$

where g_i is the gradient and η_i (Hessian) is the curvature of the loss at observation i.

3.2 2. Explicit Regularization

XGBoost adds an explicit penalty term Ω on tree complexity directly into the objective being optimized — not just via early stopping:

$$\Omega(h) = \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

where T is the number of leaves and w_j are the leaf weights (predicted values). `gamma` penalizes adding more leaves; `lambda` (L2) and `alpha` (L1) shrink the leaf weights themselves.

Key Idea

This is XGBoost's defining idea: tree complexity is *built into the loss function* being minimized, not bolted on afterwards. The algorithm literally will not add a split unless the gain outweighs the complexity penalty γ.

3.3 3. Smart Handling of Missing Values

At each split, XGBoost learns a **default direction** (left or right) for missing values, based on which direction reduces loss the most — so missing data does not need to be imputed beforehand.

3.4 Code

```
from xgboost import XGBClassifier

xgb_clf = XGBClassifier(
    n_estimators=300, learning_rate=0.05, max_depth=4,
    gamma=0.1, reg_lambda=1.0, reg_alpha=0.0,
    subsample=0.8, colsample_bytree=0.8,
    eval_metric="logloss", random_state=42
)
xgb_clf.fit(X_train, y_train)
y_pred = xgb_clf.predict(X_test)
y_proba = xgb_clf.predict_proba(X_test)[:, 1]
```

3.5 Key Hyperparameters

Parameter	What it controls
`max_depth`	Maximum depth of each tree; shallow (3–6) is typical for tabular data
`learning_rate`	Shrinkage applied to each tree's contribution
`n_estimators`	Number of boosting rounds (trees); paired with early stopping
`subsample`	Fraction of *rows* sampled per tree (like bagging, reduces overfitting)
`colsample_bytree`	Fraction of *columns* sampled per tree
`gamma`	Minimum loss reduction required to add a further split (complexity penalty)
`reg_lambda` / `reg_alpha`	L2 / L1 regularization on leaf weights

Worked Example

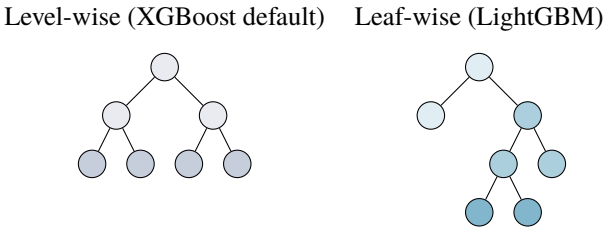
On the Titanic dataset, a shallow XGBoost (`max_depth=3`, `n_estimators=300`, `learning_rate=0.05`) typically edges out a tuned Random Forest on ROC-AUC, while training in a fraction of the time it would take a deep, unconstrained single tree to overfit the same accuracy on the training set.

4 LightGBM: Light Gradient Boosting Machine

LightGBM (Microsoft) targets the same GBM objective as XGBoost but is engineered for **speed and memory efficiency** on very large datasets, through three key innovations.

4.1 1. Leaf-wise (Best-first) Tree Growth

Most trees — including XGBoost's default — grow **level-wise**: every node at the current depth is split before moving to the next depth. LightGBM instead grows **leaf-wise**: it always splits whichever leaf gives the largest reduction in loss, regardless of depth.



Intuition

Level-wise growth is like mowing a lawn in neat, even rows regardless of where the grass is longest. Leaf-wise growth is like always attacking the single patch of longest grass first, wherever it is — it reduces loss faster per split, but can produce deeper, less-balanced trees, which is why `num_leaves` (not just depth) is the key complexity control.

Watch Out

Leaf-wise growth converges faster but overfits more easily on small datasets. LightGBM's `num_leaves` must be tuned carefully — a common rule of thumb is keeping `num_leaves` well below $2^{\texttt{max_depth}}$.

4.2 2. Histogram-Based Splitting

Instead of evaluating every possible split point on continuous features (as naive implementations do), LightGBM first buckets feature values into discrete **bins** (histograms, typically 255 by default) and only searches for splits between bins. This turns an $O(\#\text{data} \times \#\text{features})$ search into something much cheaper, and uses far less memory.

4.3 3. GOSS and EFB

- **GOSS** (Gradient-based One-Side Sampling): keeps all the training examples with *large* gradients (the ones the model is still getting wrong — most informative) but randomly subsamples the ones with small gradients (already well-fit), speeding up training with little accuracy loss.
- **EFB** (Exclusive Feature Bundling): many sparse, mutually-exclusive features (e.g. one-hot-encoded columns) can be safely bundled into a single feature, reducing the effective feature count without losing information.

4.4 Code

```
from lightgbm import LGBMClassifier

lgbm_clf = LGBMClassifier(
    n_estimators=300, learning_rate=0.05, num_leaves=15,
    max_depth=-1, subsample=0.8, colsample_bytree=0.8,
    random_state=42
)
```

```
lgbm_clf.fit(X_train, y_train)
y_pred = lgbm_clf.predict(X_test)
y_proba = lgbm_clf.predict_proba(X_test)[:, 1]
```

4.5 Key Hyperparameters

Parameter	What it controls
`num_leaves`	Main complexity control for leaf-wise trees (not `max_depth`)
`max_depth`	Optional hard depth cap; `-1` means unlimited (controlled by `num_leaves` instead)
`min_child_samples`	Minimum data points required in a leaf; higher = more conservative
`learning_rate`, `n_estimators`	Same shrinkage/rounds trade-off as GBM and XGBoost
`subsample`, `colsample_bytree`	Row / column subsampling per tree

5 XGBoost vs. LightGBM vs. Random Forest

	Random Forest	XGBoost	LightGBM
Tree building	Parallel, independent	Sequential, level-wise	Sequential, leaf-wise
Typical speed	Fast	Moderate	Very fast, large data
Memory use	Moderate–high	Moderate	Low (histogram binning)
Handles missing values	No (impute first)	Yes, built-in	Yes, built-in
Overfitting risk	Low (averaging)	Moderate (regularized)	Higher if `num_leaves` unchecked
Best suited for	Quick strong baseline, robustness	Competitions, mid-size tabular data	Very large datasets, many categorical features

Key Idea

XGBoost and LightGBM usually out-perform Random Forest on structured/tabular data when tuned properly, which is why gradient boosting methods dominate tabular ML leaderboards (e.g. Kaggle). Random Forest remains an excellent, low-effort baseline because it needs far less tuning to avoid overfitting.

6 Early Stopping and Evaluation

6.1 Why Early Stopping Matters

With enough boosting rounds, GBM/XGBoost/LightGBM will eventually start overfitting the training set. **Early stopping** monitors performance on a held-out validation set and halts training once that score stops improving for a set number of rounds (`early_stopping_rounds`), automatically picking a good `n_estimators`.

```
from sklearn.model_selection import train_test_split

X_tr, X_val, y_tr, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=42, stratify=y_train
)

xgb_clf = XGBClassifier(
    n_estimators=1000, learning_rate=0.03, max_depth=4,
    eval_metric="logloss", early_stopping_rounds=30, random_state=42
)
xgb_clf.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], verbose=False)

print("Best iteration:", xgb_clf.best_iteration)
```

6.2 Evaluating the Final Model

Use the same metrics covered for the earlier classifiers: confusion matrix, precision, recall, F1-score, and ROC-AUC on a held-out test set.

```
from sklearn.metrics import classification_report, roc_auc_score

print(classification_report(y_test, y_pred, target_names=["Died", "Survived"
    ]))
print("ROC-AUC:", roc_auc_score(y_test, y_proba))
```

Note

Both XGBoost and LightGBM expose `feature_importances_` just like `RandomForestClassifier`, so the same feature-importance bar-chart code from the Random Forest handbook works unchanged here.

7 Practice Questions

1. Explain the core difference between bagging and boosting in your own words. Why does bagging mainly reduce variance, while boosting mainly reduces bias?
2. What does it mean for GBM to "fit a tree to the negative gradient of the loss"? Walk through what happens at stage 2 if stage 1's predictions were all too low.
3. Why does lowering the learning rate usually require *increasing* `n_estimators` to reach the same accuracy?
4. Name two mechanisms XGBoost uses to explicitly control model complexity (beyond early stopping), and explain what each one penalizes.
5. Why is `num_leaves` a more important tuning knob than `max_depth` for LightGBM's leaf-wise trees?
6. Describe, at a high level, what GOSS and EFB each do to speed up LightGBM training.
7. Using the Titanic notebook, swap `RandomForestClassifier` for `XGBClassifier` with similar `n_estimators`. Compare ROC-AUC and training time between the two.
8. A model gets much higher accuracy on the training set than on a validation set as boosting rounds increase. What are two hyperparameter changes you could try to close that gap?

Try It Yourself

Train an XGBoost model with early stopping on the Titanic data. Plot the validation log-loss against boosting round. At roughly which round does the validation curve start turning back upward — and does that match the `best_iteration` XGBoost reports?